UF Herbert Wertheim College of Engineering
UNIVERSITY of FLORIDA

EEL4712 Digital Design

Lecture 20 – Verification using Randomized Testing

Rickard Ewetz
Spring 2026

Agenda

1. Accessing QuestaSim on the ECE server
2. Motivation for Randomized Testing

Advanced Verification Features Requires Paid Version of QuestaSim

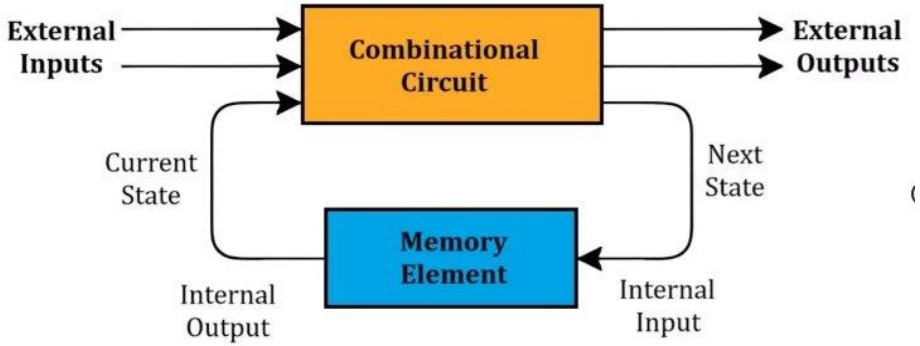
- Advanced verification features requires paid version of QuestaSim
- The paid version is available on the ECE server
- A guide for how to access the server is available on the course website (link)

1. Motivation: Verification Is About Time

- Combinational logic:
  - Correct output = function of current inputs
- Sequential logic:
  - Behavior depends on history
  - Bugs depend on when signals change
- Directed tests assume:
  - Clean ordering
  - Expected timing



Combinational Circuit

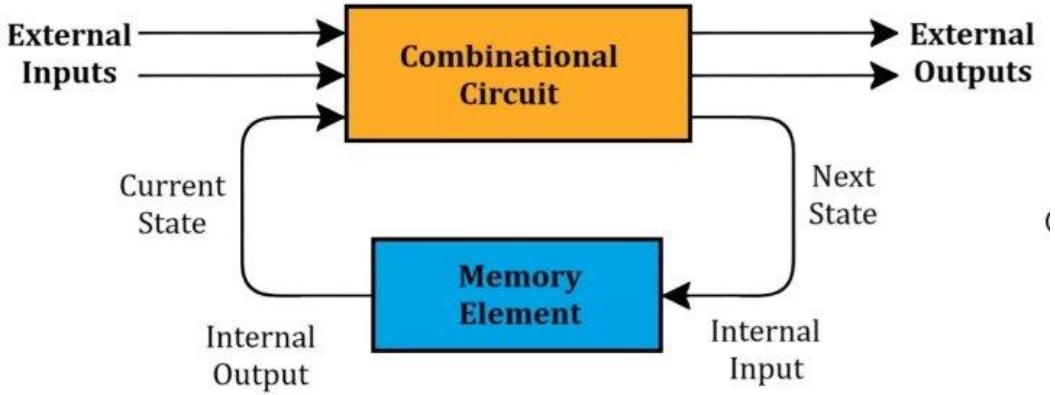


Sequential Circuit

"Testing *values* is easy; testing *sequences* is hard"

Can We Not Test Everything using Directed Testing?

- A circuit with 200 FF has 2^200 states
- Times 2^N inputs
- It is impossible to enumerate all input combinations



Sequential Circuit

Temporal Interleavings & Reachability

- Sequential logic =
  - State + transitions
- Many states are:
  - Rarely reachable
  - Never intentionally tested
- Same inputs + different timing → different outcomes
- Key idea:
  - Input ordering defines state trajectory

How do we explore these interleavings systematically?

Controlling the Solution Space and Hidden Assumptions

- Directed tests encode designer intent
- Often assume:
  - Reset is clean
  - Enables don’t glitch
  - Control signals don’t overlap
- Tests pass because assumptions hold
- Reality check:
  - Passing tests ≠ explored behavior

Why Selective Testing Does Not Always Work

- This counter is supposed to count 0,1,…, MAX, 0, 1, ...
- What is wrong with the code?

```
process(clk)
begin
   if rising_edge(clk) then
      if rst = '1' then
         cnt <= (others => '0');
      elsif en = '1' then
         cnt <= cnt + 1;
      end if;

      if cnt = MAX then
         cnt <= (others => '0');
      end if;
   end if;
end process;
```

Randomization as Sequence Exploration

- Randomization explores:
  - Ordering
  - Timing
  - Control interaction
- Deterministic ≠ thorough
- Random ≠ uncontrollable
- Introduce:
  - Per-cycle random stimulus
  - Repeatable randomness via seeds

2. Randomized Testbench: Big Picture

- Drive inputs every cycle
  - `$urandom_range`
- Check behavior every cycle
  - Random tests must be self-checking
- Measure what was exercised using Functional Coverage
  - Cover groups
  - Cover points
  - Bins

$urandom_range(max) -> [0, max]
$urandom_range(max, min) -> [min, max]

Coverage ideas =>

- Enable:
  - Asserted vs deasserted
- Reset:
  - During operation
- Control:
  - Enable toggling patterns

2.1 Why Functional Coverage Is Needed

- Random stimulus ≠ guaranteed exploration
- Simulator doesn't know your intent
- Need a way to measure what happened
- Concepts:
  - Covergroup
    - A **covergroup** is a container for coverage definitions.
  - Coverpoint
    - A **coverpoint** tracks coverage of a specific variable or expression
  - Bins
    - **Bins** divides values into categories

Details will be covered next lecture

2.2 Running the Simulation

Running in Questa

- Compile + simulate
- Set random seed
- Run long enough to accumulate coverage

2.3 Viewing Coverage Results

- Coverage summary window
- Percent coverage
- Hit vs missed bins
- Interpretation:
  - Random test did *something*
  - Coverage shows what it did *not* do
- Key insight:
  - Coverage reveals blind spots even when directed tests pass

2.4 Seeds, Debugging, and Reproducibility

- Failures tied to specific seeds
- Re-run same seed → reproduce bug
- Fix → rerun regression
- Seeds are part of the workflow

In-Class Example

- Demo Dice Example
- Next slide set has details

Summary

- Directed tests:
  - Narrow
  - Intentional
- Random tests:
  - Broad
  - Exploratory
- Coverage:
  - Connects intent to exploration